

1. Niven Number – A number divisible by the sum of its digits.

```
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), sum = 0, num = n;
        while (num > 0) {
            sum += num % 10;
            num /= 10;
        }
        if (n % sum == 0) {
            System.out.println("Niven Number");
        } else {
            System.out.println("Not a Niven Number");
        }
    }
}
```

2. Perfect Number – A number equal to the sum of its proper divisors.

```
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), sum = 0;
        for (int i = 1; i < n; i++) {
            if (n % i == 0) sum += i;
        }
        if (sum == n) {
            System.out.println("Perfect Number");
        } else {
            System.out.println("Not a Perfect Number");
        }
    }
}
```

3. Amicable Pair – Two numbers where each is the sum of the other's proper divisors.

```
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1 = sc.nextInt(), num2 = sc.nextInt(), sum1 = 0, sum2 = 0;
        for (int i = 1; i < num1; i++) {
            if (num1 % i == 0) sum1 += i;
        }
        for (int i = 1; i < num2; i++) {
            if (num2 % i == 0) sum2 += i;
        }
        if (sum1 == num2 && sum2 == num1) {
            System.out.println("Amicable Pair");
        } else {
            System.out.println("Not an Amicable Pair");
        }
    }
}
```

4. Tech Number – Even digit number which when split and squared sum equals the number.

```
import java.util.Scanner;
```

```

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt(), n = num, count = 0;
        while (n > 0) {
            count++;
            n /= 10;
        }
        if (count % 2 == 0) {
            int div = (int) Math.pow(10, count / 2);
            int first = num / div;
            int second = num % div;
            int sum = first + second;
            if (sum * sum == num) {
                System.out.println("Tech Number");
            } else {
                System.out.println("Not a Tech Number");
            }
        } else {
            System.out.println("Not a Tech Number");
        }
    }
}

```

5. Duck Number – A number containing zero but not at the beginning.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        int num2 = num;
        boolean isDuck = false;
        while (num2 > 0) {
            int digit = num2 % 10;
            if (digit == 0) isDuck = true;
            num2 /= 10;
        }
        int firstDigit = num;
        while (firstDigit >= 10) {
            firstDigit /= 10;
        }
        if (firstDigit == 0) {
            System.out.println("Not a Duck Number");
        } else if (isDuck) {
            System.out.println("Duck Number");
        } else {
            System.out.println("Not a Duck Number");
        }
    }
}

```

6. Dudney Number – A number where the sum of digits is squared repeatedly until a single digit is obtained and equals 1.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {

```

```

Scanner sc = new Scanner(System.in);
int num = sc.nextInt();
int sum = 0, n = num;
while (n > 0) {
    sum += n % 10;
    n /= 10;
}
while (sum > 9) {
    int temp = sum;
    sum = 0;
    while (temp > 0) {
        sum += temp % 10;
        temp /= 10;
    }
}
if (sum == 1) {
    System.out.println("Dudney Number");
} else {
    System.out.println("Not a Dudney Number");
}
}
}

```

7. Spy Number – A number whose sum of digits equals product of digits.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt(), sum = 0, product = 1, n = num;
        while (n > 0) {
            int d = n % 10;
            sum += d;
            product *= d;
            n /= 10;
        }
        if (sum == product) {
            System.out.println("Spy Number");
        } else {
            System.out.println("Not a Spy Number");
        }
    }
}

```

8. Armstrong Number – Sum of cube of digits equals the number (3-digit).

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt(), sum = 0, n = num;
        while (n > 0) {
            int d = n % 10;
            sum += d * d * d;
            n /= 10;
        }
        if (sum == num) {
            System.out.println("Armstrong Number");
        }
    }
}

```

```

    } else {
        System.out.println("Not an Armstrong Number");
    }
}
}

```

9. Special 2 Digit Number – Sum of digits + product of digits equals the number.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        int d1 = num / 10, d2 = num % 10;
        if (d1 + d2 + d1 * d2 == num) {
            System.out.println("Special Number");
        } else {
            System.out.println("Not a Special Number");
        }
    }
}

```

10. Palindrome Number – A number that reads same forward and backward.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt(), rev = 0, n = num;
        while (n > 0) {
            rev = rev * 10 + n % 10;
            n /= 10;
        }
        if (rev == num) {
            System.out.println("Palindrome Number");
        } else {
            System.out.println("Not a Palindrome Number");
        }
    }
}

```

11. Super Spy Number – A spy number whose digits are in arithmetic progression.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String num = sc.next();
        int sum = 0, product = 1;
        boolean ap = true;
        for (int i = 0; i < num.length(); i++) {
            int d = num.charAt(i) - '0';
            sum += d;
            product *= d;
        }
        int diff = (num.charAt(1) - '0') - (num.charAt(0) - '0');
        for (int i = 1; i < num.length() - 1; i++) {
            int d1 = num.charAt(i) - '0';
            int d2 = num.charAt(i + 1) - '0';
            if (d2 - d1 != diff) {

```

```

        ap = false;
        break;
    }
}
if (sum == product && ap) {
    System.out.println("Super Spy Number");
} else {
    System.out.println("Not a Super Spy Number");
}
}
}

```

12. Fibonacci Series up to N – Series where each term is sum of previous two, starting from 0, 1.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
    }
}

```

13. Tribonacci – Like Fibonacci but sum of previous 3 terms.

```

java
CopyEdit
import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), a = 0, b = 1, c = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int d = a + b + c;
            a = b;
            b = c;
            c = d;
        }
    }
}

```

14. Pell Series – Starts with 0, 1 and uses: $P(n) = 2 * P(n-1) + P(n-2)$.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), a = 0, b = 1;
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int c = 2 * b + a;
            a = b;
            b = c;
        }
    }
}

```

```
}  
}
```

15. Lucas Series – Like Fibonacci, but starts with 2, 1.

```
import java.util.Scanner;  
class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = sc.nextInt(), a = 2, b = 1;  
        for (int i = 0; i < n; i++) {  
            System.out.print(a + " ");  
            int c = a + b;  
            a = b;  
            b = c;  
        }  
    }  
}
```

16. Prime Number Check – Number greater than 1 with no divisors other than 1 and itself.

```
import java.util.Scanner;  
class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int num = sc.nextInt(), count = 0;  
        for (int i = 1; i <= num; i++) {  
            if (num % i == 0) count++;  
        }  
        if (count == 2) {  
            System.out.println("Prime Number");  
        } else {  
            System.out.println("Not a Prime Number");  
        }  
    }  
}
```

17. Composite Number Check – Number with more than two factors.

```
import java.util.Scanner;  
class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int num = sc.nextInt(), count = 0;  
        for (int i = 1; i <= num; i++) {  
            if (num % i == 0) count++;  
        }  
        if (count > 2) {  
            System.out.println("Composite Number");  
        } else {  
            System.out.println("Not a Composite Number");  
        }  
    }  
}
```

18. Twin Prime Check – Two prime numbers that differ by 2 (e.g., 11 and 13).

```
=  
import java.util.Scanner;  
class Main {  
    public static boolean isPrime(int num) {  
        int count = 0;
```

```

    for (int i = 1; i <= num; i++) {
        if (num % i == 0) count++;
    }
    return count == 2;
}
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int num1 = sc.nextInt(), num2 = sc.nextInt();
    if (isPrime(num1) && isPrime(num2) && Math.abs(num1 - num2) == 2) {
        System.out.println("Twin Primes");
    } else {
        System.out.println("Not Twin Primes");
    }
}
}

```

19. First N Prime Numbers – Print the first N prime numbers.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt(), count = 0, num = 2;
        while (count < n) {
            int div = 0;
            for (int i = 1; i <= num; i++) {
                if (num % i == 0) div++;
            }
            if (div == 2) {
                System.out.print(num + " ");
                count++;
            }
            num++;
        }
    }
}

```

20. Factorial of N – Product of all positive integers up to N.

```

import java.util.Scanner;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = sc.nextInt();
        int fact = 1;
        for (int i = 1; i <= num; i++) {
            fact *= i;
        }
        System.out.println(fact);
    }
}

```